



University Database System

MISY 3331

Advanced Database Systems

Instructor: Dr. Shubashini Rathina Velu

Dana Alzahid 202100121

Rafal Alqahtani 202200062

Daman Arshad 202300481

Submission Date: 16th May 2026

Table of Contents

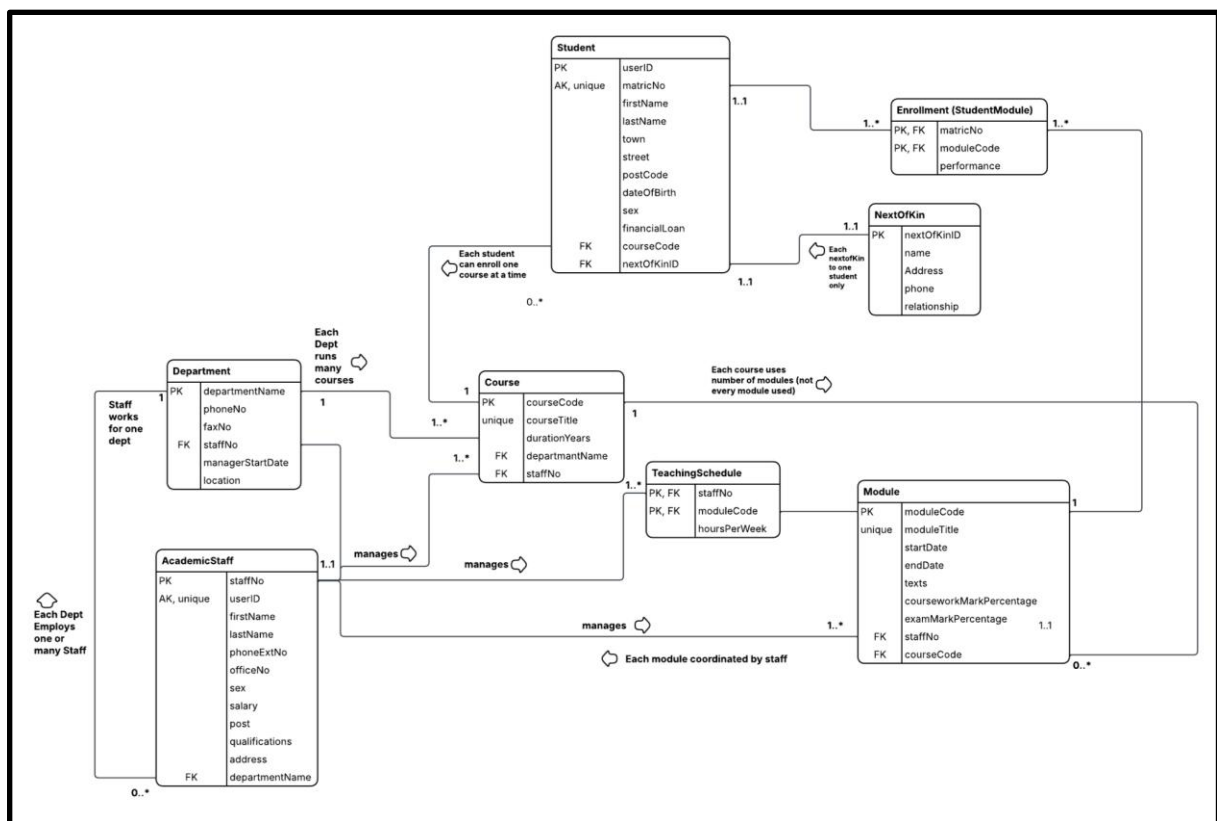
1. Introduction.....	3
2. Part 1 – Database Design.....	3
2.1 ER Diagram (UML Class Diagram)	3
2.2 Design Assumptions	4
2.3 Relational Schema.	4
2.4 Functional Dependencies	6
2.5 Normalization (Up to 3NF).....	7
2.6 Normalization of Given Relation.....	8
3. Part 2 – Database Implementation	11
3.1 Table Creation using Microsoft Access	11
3.2 Table Creation using SQL in Oracle.....	16
3.4 Sample Data using Microsoft Access	19
3.5 Sample Data using SQL	21
4. Part 3 – Query the Database	23
4.1 SQL Queries and Results	23
4.2 Views	29
4.3 Aggregated Columns	31
5. Part 4 – Database Application using Microsoft Access (Prototype).....	34
5.1 Forms	34
6. Part 5 – User Manual.....	36
6.1 System Overview	37
6.2 How to Use the System using Microsoft Access (for forms)	38
6.3. SQL Database Manual	42
7. Work Distribution.....	45
8. Part 6 – Critical Evaluation	45
9. Conclusion	49

1. Introduction

This project involves the design and implementation of a relational database system for a university. The system is intended to manage information related to courses, modules, students, academic staff, and departments. It supports both students and academic staff in accessing and managing academic data efficiently.

2. Part 1 – Database Design

2.1 ER Diagram (UML Class Diagram)



https://lucid.app/lucidchart/a720ab05-c1e9-4236-a670-1432af7510d1/edit?beaconFlowId=11CCEA13F4C44139&page=0_0&invitationId=inv_eab65a90-d5ef-479e-96a3-08e31d33c4a7#

Figure 1 – ER Diagram of the University Database

2.2 Design Assumptions

1. A student can study many modules, and a module can be studied by many students. Therefore, Enrollment (StudentModule) is used as an associative entity.
2. The student's performance is stored in Enrollment because it depends on both the student and the module.
3. An academic staff member can teach many modules, and a module may be taught by many academic staff members. Therefore, TeachingSchedule is used as an associative entity.
4. hoursPerWeek is stored in TeachingSchedule because it depends on both staff and module.
5. Since next-of-kin attributes are not unique, nextOfKinID is used as a surrogate primary key.
6. Each student has exactly one next-of-kin, so Student contains a foreign key (nextOfKinID).
7. userID is treated as an alternate key for both Student and AcademicStaff.
8. courseTitle and moduleTitle are unique and treated as alternate keys.
9. Indexes were created on foreign keys to improve performance.
10. Default value set for financialLoan (DEFAULT FALSE NOT NULL).
11. CHECK constraints added for sex, coursework/exam percentages, and hoursPerWeek.
12. Data insertion follows parent → child order to maintain referential integrity.
13. A department can exist without any academic staff assigned to it. Similarly, a department can exist without a manager assigned. This allows new departments to be created before staff are hired or a manager is appointed.

2.3 Relational Schema.

Department

Department (departmentName, phoneNo, faxNo, staffNo, managerStartDate, location)

Primary Key: departmentName

Foreign Key: staffNo → AcademicStaff(staffNo)

Course

Course (courseCode, courseTitle, durationYears, departmentName, staffNo)

Primary Key: courseCode

Alternate Key: courseTitle

Foreign Keys:

- departmentName → Department(departmentName)
- staffNo → AcademicStaff(staffNo)

Module

Module (moduleCode, moduleTitle, startDate, endDate, texts,
courseworkMarkPercentage, examMarkPercentage, **staffNo**, courseCode)

Primary Key: moduleCode

Alternate Key: moduleTitle

Foreign Keys:

- courseCode → Course(courseCode)
- staffNo → AcademicStaff(staffNo)

Student

Student (matricNo, userID, firstName, lastName, town, street, postCode, dateOfBirth,
sex, financialLoan, courseCode, **nextOfKinID**)

Primary Key: matricNo

Alternate Key: userID

Foreign Keys:

- courseCode → Course(courseCode)
- nextOfKinID → NextOfKin(nextOfKinID)

NextOfKin

NextOfKin (nextOfKinID, name, address, phone, relationship)

Primary Key: nextOfKinID

AcademicStaff

AcademicStaff (staffNo, userID, firstName, lastName, phoneExtNo, officeNo, sex,
salary, post, qualifications, address, departmentName)

Primary Key: staffNo

Alternate Key: userID

Foreign Key: departmentName → Department(departmentName)

Enrollment (StudentModule)

Enrollment(matricNo, moduleCode, performance)

Primary Key: matricNo, moduleCode

Foreign Key: matricNo references Student(matricNo)

Foreign Key: moduleCode references Module(moduleCode)

TeachingSchedule

Teaches (staffNo, moduleCode, hoursPerWeek)

Primary Key: staffNo, moduleCode

Foreign Key: staffNo references AcademicStaff(staffNo)

Foreign Key: moduleCode references Module(moduleCode)

2.4 Functional Dependencies

Department

departmentName → phoneNo, faxNo, staffNo, managerStartDate, location

Course

courseCode → courseTitle, durationYears, departmentName, staffNo

courseTitle → courseCode, durationYears, departmentName, staffNo

Module

moduleCode → moduleTitle, startDate, endDate, texts, courseworkMarkPercentage,
examMarkPercentage, courseCode, staffNo

moduleTitle → moduleCode, startDate, endDate, texts, courseworkMarkPercentage,
examMarkPercentage, courseCode, staffNo

Student

matricNo → userID, firstName, lastName, town, street, postCode, dateOfBirth, sex,
financialLoan, courseCode, nextOfKinID

$\text{userID} \rightarrow \text{matricNo}, \text{firstName}, \text{lastName}, \text{town}, \text{street}, \text{postCode}, \text{dateOfBirth}, \text{sex}, \text{financialLoan}, \text{courseCode}, \text{nextOfKinID}$

NextOfKin

$\text{nextOfKinID} \rightarrow \text{name}, \text{address}, \text{phone}, \text{relationship}$

AcademicStaff

$\text{staffNo} \rightarrow \text{userID}, \text{firstName}, \text{lastName}, \text{phoneExtNo}, \text{officeNo}, \text{sex}, \text{salary}, \text{post}, \text{qualifications}, \text{address}, \text{departmentName}$

$\text{userID} \rightarrow \text{staffNo}, \text{firstName}, \text{lastName}, \text{phoneExtNo}, \text{officeNo}, \text{sex}, \text{salary}, \text{post}, \text{qualifications}, \text{address}, \text{departmentName}$

Enrollment (StudentModule)

$\text{matricNo}, \text{moduleCode} \rightarrow \text{performance}$

TeachingSchedule

$\text{staffNo}, \text{moduleCode} \rightarrow \text{hoursPerWeek}$

2.5 Normalization (Up to 3NF)

Third Normal Form (3NF)

All relations are in Third Normal Form because they are already in 2NF (no partial dependency on composite keys) There are **no transitive dependencies**. Every non-key attribute depends only on the primary key. For example:

1. In **Student**, student details depend only on **matricNo**.
2. In **Module**, module details depend only on **moduleCode**.
3. In **AcademicStaff**, staff details depend only on **staffNo**.

Therefore, the database schema is normalized to 3NF.

2.6 Normalization of Given Relation

The given relation is:

StudentModuleFlat

(matricNo, name, sex, moduleTitle, moduleStartDate, performance, flatNo, address)

The coursework asks us to identify the functional dependencies and normalize this badly structured relation into 3NF.

Functional Dependencies

1. $\text{matricNo} \rightarrow \text{name, sex, flatNo}$
2. $\text{flatNo} \rightarrow \text{address}$
3. $\text{moduleTitle} \rightarrow \text{moduleStartDate}$
4. $\text{matricNo, moduleTitle} \rightarrow \text{performance}$

Unnormalized Relation

StudentModuleFlat

(matricNo, name, sex, moduleTitle, moduleStartDate, performance, flatNo, address)

Problem:

1. Student details are repeated.
2. Module details are repeated.
3. Address depends on flatNo, not directly on matricNo.
4. Performance depends on both student and module.

First Normal Form (1NF)

The relation is in 1NF because all values are atomic and there are no repeating groups.

StudentModuleFlat

(matricNo, name, sex, moduleTitle, moduleStartDate, performance, flatNo, address)

Primary Key: matricNo, moduleTitle

Second Normal Form (2NF)

To reach 2NF, remove partial dependencies.

Partial dependencies:

1. $\text{matricNo} \rightarrow \text{name, sex, flatNo}$
2. $\text{moduleTitle} \rightarrow \text{moduleStartDate}$

So we split the relation into:

Student

(matricNo , name , sex , flatNo)

Primary Key: matricNo

Module

(moduleTitle , moduleStartDate)

Primary Key: moduleTitle

Enrollment (StudentModule)

(matricNo , moduleTitle , performance)

Primary Key: matricNo , moduleTitle

Foreign Key: matricNo references $\text{Student}(\text{matricNo})$

Foreign Key: moduleTitle references $\text{Module}(\text{moduleTitle})$

Third Normal Form (3NF)

To reach 3NF, remove transitive dependencies.

In Student:

$\text{matricNo} \rightarrow \text{flatNo}$

$\text{flatNo} \rightarrow \text{address}$

So address depends on flatNo, not directly on matricNo. This is a transitive dependency.

Split into:

Student

(matricNo, name, sex, flatNo)

Primary Key: matricNo

Foreign Key: flatNo references Flat(flatNo)

Flat

(flatNo, address)

Primary Key: flatNo

Module

(moduleTitle, moduleStartDate)

Primary Key: moduleTitle

Enrollment (StudentModule)

(matricNo, moduleTitle, performance)

Primary Key: matricNo, moduleTitle

Foreign Key: matricNo references Student(matricNo)

Foreign Key: moduleTitle references Module(moduleTitle)

Final 3NF Relations

1. **Student** (matricNo, name, sex, flatNo)
2. **Flat** (flatNo, address)
3. **Module** (moduleTitle, moduleStartDate)
4. **Enrollment** (matricNo, moduleTitle, performance)

3. Part 2 – Database Implementation

3.1 Table Creation using Microsoft Access

Department table

```
CREATE TABLE Department (
    departmentName TEXT PRIMARY KEY,
    phoneNo TEXT,
    faxNo TEXT,
    managerStaffNo TEXT,
    dateStartedManaging DATE,
    location TEXT
);
```

Field Name		Data Type	Description (Optional)
departmentName		Short Text	
phoneNo		Short Text	
faxNo		Short Text	
managerStaffNo		Short Text	
dateStartedManaging		Date/Time	
location		Short Text	

Field Properties

General

Lookup

Field Size	255
Format	
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Allow Zero Length	Yes
Indexed	Yes (No Duplicates)
Unicode Compression	No
IME Mode	No Control
IME Sentence Mode	None
Text Align	General

Course table

```
CREATE TABLE Course (
  courseCode TEXT PRIMARY KEY,
  courseTitle TEXT,
  durationYears INTEGER,
  departmentName TEXT,
  courseLeaderStaffNo TEXT
);
```


Field Name	Data Type	Description (Optional)
moduleCode	Short Text	
moduleTitle	Short Text	
startDate	Date/Time	
endDate	Date/Time	
texts	Short Text	
courseworkMarkPercentage	Number	
examMarkPercentage	Number	
courseCode	Short Text	
coordinatorStaffNo	Short Text	

Field Properties

General	
Field Size	255
Format	
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Allow Zero Length	Yes
Indexed	Yes (No Duplicates)
Unicode Compression	No
IME Mode	No Control
IME Sentence Mode	None
Text Align	General

Student table

```

CREATE TABLE Student (
  matricNo TEXT PRIMARY KEY,
  userID TEXT,
  firstName TEXT,
  lastName TEXT,
  town TEXT,
  street TEXT,
  postCode TEXT,
  dateOfBirth DATE,
  sex TEXT,
  financialLoan YESNO,
  courseCode TEXT
);

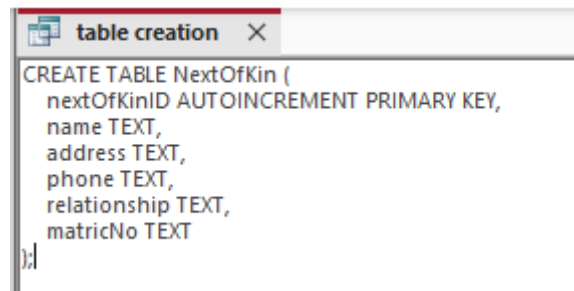
```

Field Name	Data Type	Description (Optional)
matricNo	Short Text	
userID	Short Text	
firstName	Short Text	
lastName	Short Text	
town	Short Text	
street	Short Text	
postCode	Short Text	
dateOfBirth	Date/Time	
sex	Short Text	
financialLoan	Yes/No	
courseCode	Short Text	

Field Properties

General	
Field Size	255
Format	
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Allow Zero Length	Yes
Indexed	Yes (No Duplicates)
Unicode Compression	No
IME Mode	No Control
IME Sentence Mode	None
Text Align	General

NextOfKin table

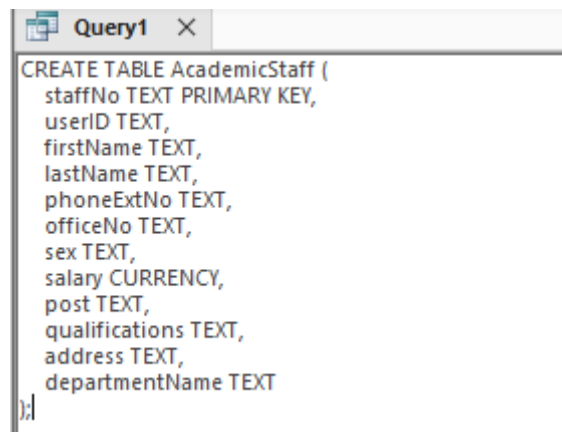


Field Name	Data Type	Description (Optional)
nextOfKinID	AutoNumber	
name	Short Text	
address	Short Text	
phone	Short Text	
relationship	Short Text	
matricNo	Short Text	

Field Properties

General	Lookup
Field Size	Long Integer
New Values	Increment
Format	
Caption	
Indexed	Yes (No Duplicates)
Text Align	General

Academic Staff table



Field Name	Data Type	Description (Optional)
userID	Short Text	
firstName	Short Text	
lastName	Short Text	
phoneExtNo	Short Text	
officeNo	Short Text	
sex	Short Text	
salary	Currency	
post	Short Text	
qualifications	Short Text	
address	Short Text	
departmentName	Short Text	

General	Lookup
Field Size	255
Format	
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Allow Zero Length	Yes
Indexed	Yes (No Duplicates)
Unicode Compression	No
IME Mode	No Control
IME Sentence Mode	None
Text Align	General

Student Modules table

```
CREATE TABLE StudentModule (
    matricNo TEXT,
    moduleCode TEXT,
    performance INTEGER,
    PRIMARY KEY (matricNo, moduleCode)
);
```

Field Name	Data Type	Description (Optional)
matricNo	Short Text	
moduleCode	Short Text	
performance	Number	

General	Lookup
Field Size	255
Format	
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Allow Zero Length	Yes
Indexed	Yes (Duplicates OK)
Unicode Compression	No
IME Mode	No Control
IME Sentence Mode	None
Text Align	General

TeachingSchedule

```
CREATE TABLE Teaches (
    staffNo TEXT,
    moduleCode TEXT,
    hoursPerWeek INTEGER,
    PRIMARY KEY (staffNo, moduleCode)
);
```

[illegible]

Figure 2 – Example Table Structure

3.2 Table Creation using SQL in Oracle

Department table -

```
1 CREATE TABLE Department
2  ( departmentName varchar(25) PRIMARY KEY,
3    phoneNo int,
4    faxNo int,
5    managerStartDate date,
6    location varchar(15));
7
```

Adding staff Foreign Key

```
73 ALTER TABLE Department ADD (staffNo varchar(10));
74 ALTER TABLE Department ADD CONSTRAINT fk_staffNo FOREIGN KEY (staffNo)
75 REFERENCES AcademicStaff(staffNo);
76
77
```

Course table

```
9 CREATE TABLE Course
10 ( courseCode varchar(25) PRIMARY KEY,
11   courseTitle varchar(25),
12   durationYears date
13 )
```

```

78 ALTER TABLE Course ADD (staffNo varchar(10));
79 ALTER TABLE Course ADD (departmentName varchar(25));
80 ALTER TABLE Course ADD CONSTRAINT fk_CoursestaffNo FOREIGN KEY (staffNo)
81 REFERENCES AcademicStaff(staffNo);
82 ALTER TABLE Course ADD CONSTRAINT fk_CourseDep FOREIGN KEY (departmentName)
83 REFERENCES Department(departmentName);

```

Module table

```

15  ✓ CREATE TABLE Module (
16      moduleCode varchar(25) PRIMARY KEY,
17      moduleTitle int,
18      startDate date,
19      endDate date,
20      texts varchar(50),
21      courseworkMarkPercentage int,
22      examMarkPercentage int);
23
85 ALTER TABLE Module ADD (staffNo varchar(10));
86 ALTER TABLE Module ADD (courseCode varchar(25));
87 ALTER TABLE Module ADD CONSTRAINT fk_modstaffNo FOREIGN KEY (staffNo)
88 REFERENCES AcademicStaff(staffNo);
89 ALTER TABLE Module ADD CONSTRAINT fk_modCourse FOREIGN KEY (courseCode)
90 REFERENCES Course(courseCode);

```

Student table

```

24  ✓ CREATE TABLE Student (
25      matcricNo varchar(10) PRIMARY KEY,
26      userID int UNIQUE,
27      firstName varchar(20),
28      lastName varchar(20),
29      town varchar(20),
30      street varchar(20),
31      postCode varchar(10),
32      dateOfBirth date,
33      sex char,
34      financialLoan BOOLEAN DEFAULT FALSE NOT NULL);

92 ALTER TABLE Student ADD (nextOfKinID int);
93 ALTER TABLE Student ADD (courseCode varchar(25));
94 ALTER TABLE Student ADD CONSTRAINT fk_stukin FOREIGN KEY (nextOfKinID)
95 REFERENCES NextOfKin(nextOfKinID);
96 ALTER TABLE Student ADD CONSTRAINT fk_stuCourse FOREIGN KEY (courseCode)
97 REFERENCES Course(courseCode);

```

NextOfKin table

```
36 CREATE TABLE NextOfKin
37 (
38     nextOfKinID int PRIMARY KEY,
39     name varchar(20),
40     address varchar(50),
41     phone int,
42     relationship varchar(10)
43 );
```

Academic Staff table

```
45 CREATE TABLE academicStaff (
46     staffNo varchar(10) PRIMARY KEY,
47     userID varchar(10) UNIQUE,
48     firstName varchar(20),
49     lastName varchar(20),
50     phoneExtNo int,
51     officeNo varchar(5),
52     sex char,
53     salary NUMBER(10,2),
54     post varchar(10),
55     qualifications varchar(25),
56     address varchar(30),
57     departmentName varchar(25) REFERENCES Department(departmentName)
```

Enrollment (StudentModule)

```
60 CREATE TABLE Enrollment (
61     matricNo int REFERENCES Student(matricNo),
62     moduleCode varchar(25) REFERENCES Module(moduleCode),
63     PRIMARY KEY (matricNo,moduleCode)
64 );
```

TeachingSchedule

```
66 CREATE TABLE TeachingSchedule (
67     staffNo varchar(10) REFERENCES AcademicStaff(staffNo),
68     moduleCode varchar(25) REFERENCES Module(moduleCode),
69     PRIMARY KEY (staffNo,moduleCode),
70     hoursPerWeek int
71 );
```

Fixing minor errors:

```

99  -----Fixing errors
100 ALTER TABLE Course MODIFY (durationYears NUMBER(2,1));
101 ALTER TABLE Module MODIFY (moduleTitle varchar(100));
102 ALTER TABLE Enrollment ADD performance varchar(4) CHECK (performance IN ('Pass', 'Fail'));

```

Adding check constraints

```

104  -----adding checks
105  ALTER TABLE AcademicStaff ADD CONSTRAINT chk_sex_academic CHECK (sex IN ('M', 'F'));
106  ALTER TABLE Student ADD CONSTRAINT chk_sex_student CHECK (sex IN ('M', 'F'));
107  ALTER TABLE Module ADD CONSTRAINT chk_coursework_percent CHECK (courseworkMarkPercentage BETWEEN 0 AND 100);
108  ALTER TABLE Module ADD CONSTRAINT chk_exam_percent CHECK (examMarkPercentage BETWEEN 0 AND 100);
109  ALTER TABLE TeachingSchedule ADD CONSTRAINT chk_hours_positive CHECK (hoursPerWeek > 0 AND hoursPerWeek <= 168);

```

Adding Cardinalities

```

323  -----Adding Cardinalities
324
325  -----NOT NULL Constraints
326
327  -- Course must have modules (1..*)
328  ALTER TABLE Module MODIFY (courseCode NOT NULL);
329
330  -- Module must have coordinator (1..*)
331  ALTER TABLE Module MODIFY (staffNo NOT NULL);
332
333  -- Student must be enrolled in a course
334  ALTER TABLE Student MODIFY (courseCode NOT NULL);
335
336  -- Student must have next of kin
337  ALTER TABLE Student MODIFY (nextOfKinID NOT NULL);
338
339  -----UNIQUE Constraints
340
341  ALTER TABLE Course ADD CONSTRAINT uk_courseTitle UNIQUE (courseTitle);
342  ALTER TABLE Module ADD CONSTRAINT uk_moduleTitle UNIQUE (moduleTitle);
343
344  -- Each next of kin belongs to one student only
345  ALTER TABLE Student ADD CONSTRAINT uk_student_nextofkin UNIQUE (nextOfKinID);
346  |

```

3.4 Sample Data using Microsoft Access

Department Table

Department							
	department	phoneNo	faxNo	managerSta	dateStarted	location	Click to Add
+	Business	012345678	012345679	S003	1/1/2022	Jeddah	
+	CIS	011234567	011234568	S001	2/1/2023	Riyadh	
+	Engineering	013456789	013456780	S003	5/1/2021	Dammam	
*							

Course Table

Course	courseCode	courseTitle	durationYea	department	courseLeade	Click to Add
+	C001	Computer Scie	4	CIS	S002	
+	C002	Business Admi	4	Business	S003	
+	C003	Software Engin	4	CIS	S005	
+	C004	Mechanical Eng	4	Engineering	S003	
*						

Module Table

Module	moduleCode	moduleTitle	startDate	endDate	texts	courseworkI	examMarkPi	courseCode	coordinator	Click to Add
+	M001	Database Syste	1/1/2024	6/1/2024	SQL Book	40	60	C001	S002	
+	M002	Multi Media	1/1/2024	6/1/2024	Media Book	50	50	C001	S002	
+	M003	Programming			Coding Book	30	70	C003	S005	
+	M004	Marketing			Marketing Boo	40	60	C002	S003	
+	M005	Thermodynam			Physics Book	50	50	C004	S003	
*										

Student Table

Student	matricNo	userID	firstName	lastName	town	street	postCode	dateOfBirth	sex	financialLoa	courseCode	Click to Add
+	ST01	stu01	Ahmed	Al Faisal	Riyadh	Street 1	11564	5/1/2002	M	☒	C001	
+	ST02	stu02	Sara	Al Zahrani	Jeddah	Street 2	21577	7/10/2003	F	☐	C001	
+	ST03	stu03	Hassan	Al Ghamdi	Dammam	Street 3	31411		M	☒	C002	
+	ST04	stu04	Noura	Al Harbi	Makkah	Street 4	24236		F	☐	C003	
+	ST05	stu05	Yousef	Al Otaibi	Riyadh	Street 5	11564		M	☒	C004	
*										☐		

NextOfKin Table

NextOfKin	nextOfKinID	name	address	phone	relationship	matricNo	Click to Add
	1	Ali	Riyadh	0501234567	Father	ST01	
	2	Fatima	Jeddah	0507654321	Mother	ST02	
	3	Saad	Dammam	0501112222	Father	ST03	
	4	Maha	Makkah	0503334444	Mother	ST04	
	5	Ibrahim	Riyadh	0505556666	Father	ST05	
*		(New)					

AcademicStaff Table

staffNo	userID	firstName	lastName	phoneExtNc	officeNo	sex	salary	post	qualification	address	department	Click to
S001	u001	Fahad	Al Saud	101	A12	M	\$12,000.00	Manager	PhD	Riyadh	CIS	
S002	u002	Aisha	Al Harbi	102	B10	F	\$10,000.00	Lecturer	PhD	Jeddah	CIS	
S003	u003	Omar	Al Qahtani	103	C05	M	\$9,000.00	Lecturer	Masters	Dammam	Business	
S004	u004	Khalid	Al Otaibi	104	D11	M	\$11,000.00	Manager	PhD	Riyadh	Business	
S005	u005	Sara	Al Shammari	105	E02	F	\$9,500.00	Lecturer	Masters	Makkah	CIS	

Enrollment Table

matricNo	moduleCode	performance	Click to Add
ST01	M001	85	
ST02	M002	90	
ST03	M004	75	
ST04	M003	88	
ST05	M005	80	

TeachingSchedule Table

staffNo	moduleCode	hoursPerWe	Click to Add
S002	M001	8	
S002	M002	6	
S003	M004	5	
S003	M005	9	
S005	M003	7	

3.5 Sample Data using SQL

Department Table

```

119 -- 2. AcademicStaff (references Department)
120 INSERT INTO AcademicStaff VALUES ('S001', 'U001', 'John', 'Smith', 123, 'A101', 'M', 55000.00, 'Mana
121 INSERT INTO AcademicStaff VALUES ('S002', 'U002', 'Jane', 'Doe', 124, 'A102', 'F', 60000.00, 'Manage
122 INSERT INTO AcademicStaff VALUES ('S003', 'U003', 'Bob', 'Brown', 125, 'A103', 'M', 45000.00, 'Lectu
123 INSERT INTO AcademicStaff VALUES ('S004', 'U004', 'Alice', 'White', 126, 'A104', 'F', 42000.00, 'Lec
124 INSERT INTO AcademicStaff VALUES ('S005', 'U005', 'Charlie', 'Black', 127, 'A105', 'M', 48000.00, 'L
125

110 -----Inserting Data
111 -- 1. First: Tables with NO foreign keys (parents)
112 INSERT INTO Department VALUES ('CIS', 1234, 5678, '01-JAN-2020', 'E Block', NULL);
113 INSERT INTO Department VALUES ('Business', 2345, 6789, '15-MAR-2019', 'A Block', NULL);
114 INSERT INTO Department VALUES ('Engineering', 3456, 7890, '10-JUN-2021', 'B Block', NULL);
115 INSERT INTO Department VALUES ('Humanities', 4567, 8901, '20-AUG-2020', 'C Block', NULL);
116 INSERT INTO Department VALUES ('Science', 5678, 9012, '05-MAY-2022', 'E Block', NULL);
117

```

Academic Staff Table

```
127 -- 3. Update Department managers (FK to AcademicStaff)
128 UPDATE Department SET staffNo = 'S001' WHERE departmentName = 'CIS';
129 UPDATE Department SET staffNo = 'S002' WHERE departmentName = 'Business';
130 UPDATE Department SET staffNo = 'S005' WHERE departmentName = 'Engineering';
131 UPDATE Department SET staffNo = 'S004' WHERE departmentName = 'Humanities';
132 UPDATE Department SET staffNo = 'S003' WHERE departmentName = 'Science';
133
```

NextOfKin Table

```
134 -- 4. NextOfKin (no dependencies)
135 INSERT INTO NextOfKin VALUES (1, 'Mary Smith', '5 Elm St', 111222, 'Mother');
136 INSERT INTO NextOfKin VALUES (2, 'Tom Doe', '6 Maple Ave', 333444, 'Father');
137 INSERT INTO NextOfKin VALUES (3, 'Lisa Brown', '7 Pine St', 555666, 'Sister');
138 INSERT INTO NextOfKin VALUES (4, 'Paul White', '8 Oak Rd', 777888, 'Brother');
139 INSERT INTO NextOfKin VALUES (5, 'Ann Black', '9 Cedar Ave', 999000, 'Spouse');
140
```

Course Table

```
141 -- 5. Course (references Department and AcademicStaff)
142 INSERT INTO Course VALUES ('PgDIT', 'Pg Diploma IT', 1.0, 'S001', 'CIS');
143 INSERT INTO Course VALUES ('MScCS', 'MSc Computer Science', 2.0, 'S003', 'CIS');
144 INSERT INTO Course VALUES ('BScIT', 'BSc IT', 3.0, 'S004', 'Business');
145 INSERT INTO Course VALUES ('MBA', 'Masters Business Admin', 1.5, 'S002', 'Business');
146 INSERT INTO Course VALUES ('BScEng', 'BSc Engineering', 3.0, 'S005', 'Engineering');
147
```

Module Table

```
148 -- 6. Module (references AcademicStaff and Course)
149 INSERT INTO Module VALUES ('M101', 'Multi Media', '01-SEP-2025', '15-DEC-2025', 'Book1', 40, 60, 'S001');
150 INSERT INTO Module VALUES ('M102', 'Databases', '01-SEP-2025', '15-DEC-2025', 'Book2', 50, 50, 'S001');
151 INSERT INTO Module VALUES ('M103', 'Programming', '01-FEB-2026', '30-MAY-2026', 'Book3', 30, 70, 'S001');
152 INSERT INTO Module VALUES ('M104', 'Networking', '01-SEP-2025', '15-DEC-2025', 'Book4', 40, 60, 'S001');
153 INSERT INTO Module VALUES ('M105', 'Cyber Security', '01-FEB-2026', '30-MAY-2026', 'Book5', 50, 50, 'S001');
154
```

Student Table

```
54
55 -- 7. Student (references NextOfKin and Course)
56 INSERT INTO Student VALUES ('STU001', 1001, 'Tom', 'Jones', 'London', '1 High St', 'AB1 2CD', '15-MAR-2025');
57 INSERT INTO Student VALUES ('STU002', 1002, 'Sarah', 'Lee', 'London', '2 Low St', 'AB2 3DE', '20-JUN-2025');
58 INSERT INTO Student VALUES ('STU003', 1003, 'Mike', 'Wong', 'Bristol', '3 Hill Rd', 'BC3 4EF', '10-JUL-2025');
59 INSERT INTO Student VALUES ('STU004', 1004, 'Emma', 'Davis', 'Birmingham', '4 Queen St', 'CD4 5FG', '01-SEP-2025');
60 INSERT INTO Student VALUES ('STU005', 1005, 'James', 'Wilson', 'Manchester', '5 King Rd', 'DE5 6GH', '15-DEC-2025');
61
```

Enrollment Table

```
162 -- 8. Enrollment (references Student and Module)
163 INSERT INTO Enrollment VALUES (1001, 'M101', 'Pass');
164 INSERT INTO Enrollment VALUES (1001, 'M102', 'Pass');
165 INSERT INTO Enrollment VALUES (1002, 'M101', 'Fail');
166 INSERT INTO Enrollment VALUES (1002, 'M102', 'Pass');
167 INSERT INTO Enrollment VALUES (1003, 'M103', 'Pass');
```

Teaching Schedule Table

```
168
169 -- 9. TeachingSchedule (references AcademicStaff and Module)
170 INSERT INTO TeachingSchedule VALUES ('S003', 'M101', 8);
171 INSERT INTO TeachingSchedule VALUES ('S001', 'M102', 5);
172 INSERT INTO TeachingSchedule VALUES ('S004', 'M103', 10);
173 INSERT INTO TeachingSchedule VALUES ('S003', 'M104', 7);
174 INSERT INTO TeachingSchedule VALUES ('S005', 'M105', 9);
175
```

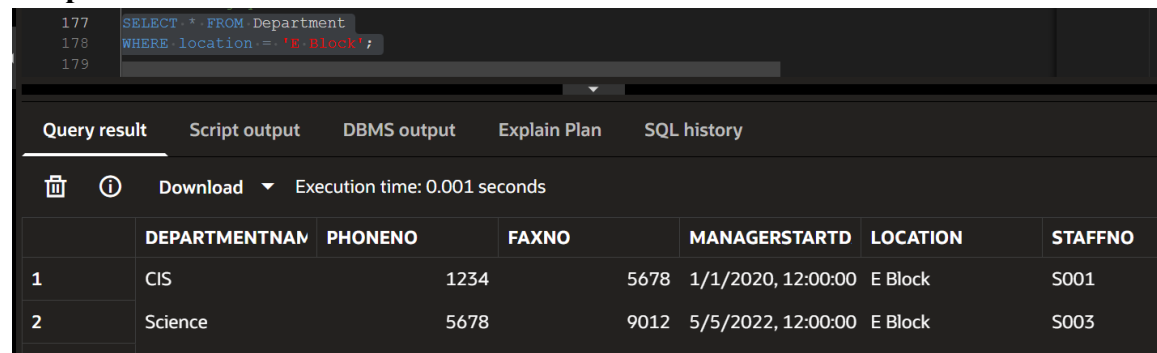
4. Part 3 – Query the Database

4.1 SQL Queries and Results

Query (a): Departments in E Block

```
SELECT * FROM Department
WHERE location = 'E Block';
```

Output:



The screenshot shows a SQL query execution interface. The query is: `SELECT * FROM Department WHERE location = 'E Block';`. The results are displayed in a table with 7 columns: DEPARTMENTNAM, PHONENO, FAXNO, MANAGERSTARTD, LOCATION, and STAFFNO. The results show two rows: 1. CIS, 1234, 5678, 1/1/2020, 12:00:00, E Block, S001. 2. Science, 5678, 9012, 5/5/2022, 12:00:00, E Block, S003.

	DEPARTMENTNAM	PHONENO	FAXNO	MANAGERSTARTD	LOCATION	STAFFNO
1	CIS	1234	5678	1/1/2020, 12:00:00	E Block	S001
2	Science	5678	9012	5/5/2022, 12:00:00	E Block	S003

Explanation:

This query retrieves all columns for departments whose location column contains the exact value 'E Block'.

Query (b): Modules in PgDIT course

```
SELECT moduleTitle, startDate, endDate
FROM Module
WHERE courseCode= 'PgDIT';
```

Output:

```
180 SELECT moduleTitle, startDate, endDate
181 FROM Module
182 WHERE courseCode= 'PgDIT';
183
```

	Query result	Script output	DBMS output	Explain Plan	SQL history
	Download Execution time: 0.001 seconds				
	MODULETITLE	STARTDATE	ENDDATE		
1	Multi Media	9/1/2025, 12:00:00	12/15/2025, 12:00:00		
2	Databases	9/1/2025, 12:00:00	12/15/2025, 12:00:00		

Explanation:

This query selects the module title, start date, and end date for all modules belonging to the course with the code 'PgDIT'. Note: Your schema uses endDate, not moduleEndDate.

Query (c): Female managers

```
SELECT firstName, lastName, address, salary
FROM AcademicStaff
WHERE sex='F' AND post='Manager';
```

Output:

```
184 SELECT firstName, lastName, address, salary
185 FROM AcademicStaff
186 WHERE sex = 'F' AND post = 'Manager';
```

	Query result	Script output	DBMS output	Explain Plan	SQL history
	Download Execution time: 0.006 seconds				
	FIRSTNAME	LASTNAME	ADDRESS	SALARY	
1	Jane	Doe	20 Main St	60000	

Explanation:

This query retrieves the first name, last name, address, and salary of academic staff whose sex is female ('F') and whose position/post title is exactly 'Manager'.

Query (d): Lecturers holding a PhD Degree

```
SELECT firstName, lastName, sex, salary
FROM AcademicStaff
WHERE post = 'Lecturer' AND qualifications='PhD'
```

Output:

```
188 SELECT firstName, lastName, sex, salary
189 FROM AcademicStaff
190 WHERE post='Lecturer' AND qualifications='PhD';
191
```

	FIRSTNAME	LASTNAME	SEX	SALARY
1	Bob	Brown	M	45000
2	Alice	White	F	42000

Explanation:

This query filters the AcademicStaff table to find staff with the post 'Lecturer' and the exact qualification 'PhD', then displays their name, sex, and salary.

Query (e): CIS Department Employees

```
SELECT lastName, post, qualifications
FROM AcademicStaff
WHERE departmentName='CIS';
```

Output:

```
192 SELECT lastName, post, qualifications
193 FROM AcademicStaff
194 WHERE departmentName='CIS';
```

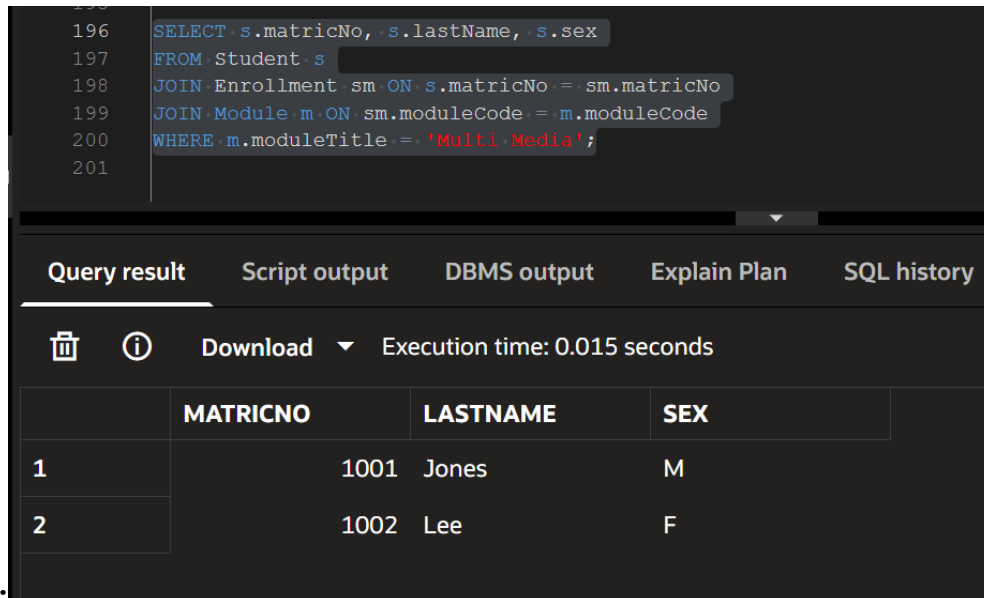
	LASTNAME	POST	QUALIFICATIONS
1	Smith	Manager	PhD
2	Doe	Manager	MSc
3	Brown	Lecturer	PhD

Explanation:

This query selects the name, job title, and qualifications of all academic staff who work in the department named 'CIS'.

Query (f): Multi Media Module Students

```
SELECT s.matricNo, s.lastName, s.sex
FROM Student s
JOIN Enrollment sm ON s.matricNo = sm.matricNo
JOIN Module m ON sm.moduleCode = m.moduleCode
WHERE m.moduleTitle = 'Multi Media';
```



The screenshot shows a SQL IDE interface. The top pane displays the SQL query for finding students enrolled in the 'Multi Media' module. The bottom pane shows the query results in a table with columns MATRICNO, LASTNAME, and SEX. The results show two students: Jones (matricNo 1001, sex M) and Lee (matricNo 1002, sex F). The interface also includes tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. A 'Download' button and 'Execution time: 0.015 seconds' are visible above the results table.

	MATRICNO	LASTNAME	SEX
1	1001	Jones	M
2	1002	Lee	F

Output:

Explanation:

This query uses two JOINS to connect Student → StudentModule → Module. It finds all students enrolled in the module titled 'Multi Media' and displays their matriculation number, name, and sex.

Query (g): Academic Staff with higher than Average Salary

```
SELECT staffNo, lastName, post, sex
FROM AcademicStaff
WHERE salary > (SELECT AVG(salary) FROM AcademicStaff)
```

Output:

```
202 SELECT staffNo, lastName, post, sex
203 FROM AcademicStaff
204 WHERE salary > (SELECT AVG(salary) FROM AcademicStaff)
205
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▾ Execution time: 0.009 seconds				
	STAFFNO	LASTNAME	POST	SEX
1	S001	Smith	Manager	M
2	S002	Doe	Manager	F

Explanation:

The subquery (SELECT AVG(salary)...) calculates the average salary across all academic staff. The main query then finds and returns staff whose individual salary is greater than that calculated average.

Query (h): Courses with more than 10 students

```
SELECT c.courseTitle, COUNT(s.matricNo) AS NumberOfStudents
FROM Course c
JOIN Student s ON c.courseCode = s.courseCode
GROUP BY c.courseCode, c.courseTitle
HAVING COUNT(s.matricNo) > 10;
```

Output:

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▾ Execution time: 0.029 seconds				
COURSETITLE	NUMBEROFSTUDE	No items to display.		

No items as no course has more than 10 students as this is running only on a small sample data.

Explanation:

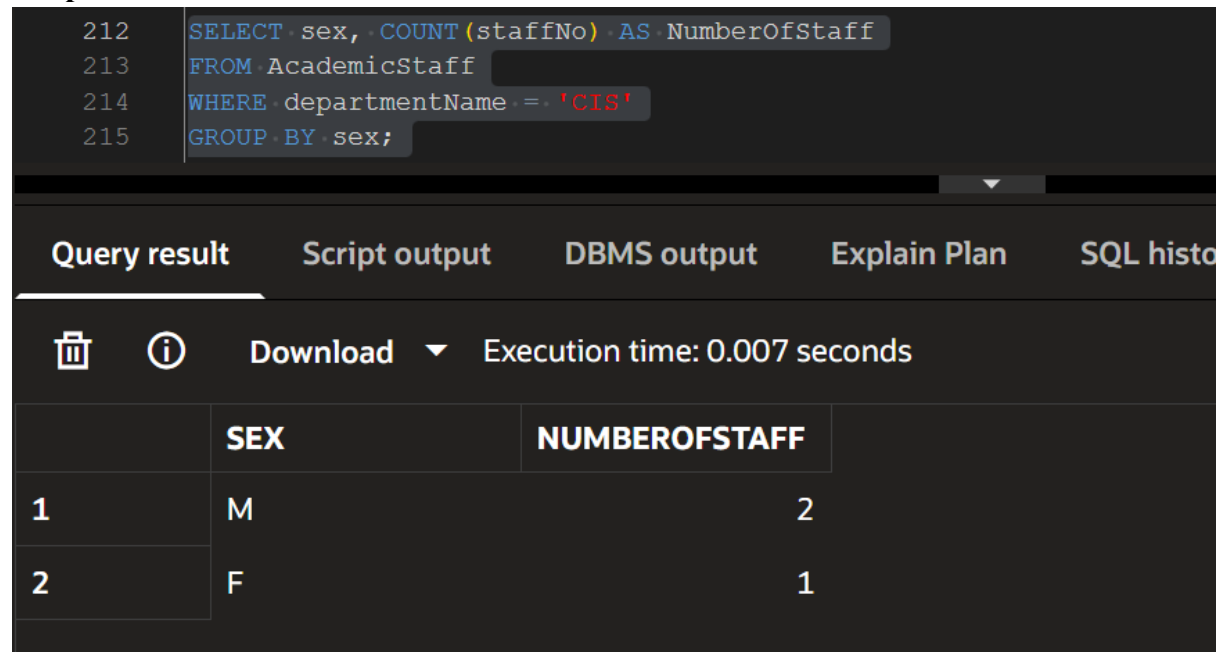
This query joins Course and Student, groups the results by each course, and counts the

students per group. The HAVING clause then filters to show only those courses with a student count greater than 10.

Query (i): Number of staff in CIS department by gender

```
SELECT sex, COUNT(staffNo) AS NumberOfStaff
FROM AcademicStaff
WHERE departmentName = 'CIS'
GROUP BY sex;
```

Output:



The screenshot shows a SQL query execution interface. The query is displayed in a text area with line numbers 212 to 215. Below the query, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Query result' tab is selected, showing a table with two columns: 'SEX' and 'NUMBEROFSTAFF'. The table has two rows: one for 'M' with a count of 2, and one for 'F' with a count of 1. The execution time is 0.007 seconds.

```
212 SELECT sex, COUNT(staffNo) AS NumberOfStaff
213 FROM AcademicStaff
214 WHERE departmentName = 'CIS'
215 GROUP BY sex;
```

	SEX	NUMBEROFSTAFF
1	M	2
2	F	1

Explanation:

This query groups staff from the 'CIS' department by their sex ('M' or 'F') and uses COUNT(staffNo) to calculate how many male and female staff work in that department.

Query (j): Academic staff with more than 6 teaching hours

```
SELECT DISTINCT a.lastName, m.moduleTitle, t.hoursPerWeek
FROM AcademicStaff a
JOIN TeachingSchedule t ON a.staffNo = t.staffNo
JOIN Module m ON t.moduleCode=m.moduleCode
WHERE t.hoursPerWeek > 6;
```

Output:

217	SELECT DISTINCT a.lastName, m.moduleTitle, t.hoursPerWeek
218	FROM AcademicStaff a
219	JOIN TeachingSchedule t ON a.staffNo = t.staffNo
220	JOIN Module m ON t.moduleCode = m.moduleCode
221	WHERE t.hoursPerWeek > 6;
222	

Query result	Script output	DBMS output	Explain Plan	SQL history
--------------	---------------	-------------	--------------	-------------

		Download	▼	Execution time: 0.028 seconds
--	--	----------	---	-------------------------------

	LASTNAME	MODULETITLE	HOURSPERWEEK
1	Black	Cyber Security	9
2	Brown	Multi Media	8
3	Brown	Networking	7
4	White	Programming	10

Explanation:

This query joins AcademicStaff, TeachingSchedule, and Module to find staff who teach more than 6 hours per week. It displays their last name, the module title, and the number of hours taught.

4.2 Views

View 1: Department Summary

Justification:

Justification: Provides quick access to department details with manager names without joining tables each time. Useful for admin reporting.

225	CREATE VIEW DepartmentSummary AS
226	SELECT d.departmentName, d.location, a.firstName ' ' a.lastName AS managerName, d.managerStart
227	FROM Department d
228	JOIN AcademicStaff a ON d.staffNo = a.staffNo;
229	
230	Select * From DepartmentSummary;

Query result	Script output	DBMS output	Explain Plan	SQL history
--------------	---------------	-------------	--------------	-------------

		Download	▼	Execution time: 0.016 seconds
--	--	----------	---	-------------------------------

	DEPARTMENTNAM	LOCATION	MANAGERNAME	MANAGERSTARTD
1	Engineering	B Block	Charlie Black	6/10/2021, 12:00:00
2	CIS	E Block	John Smith	1/1/2020, 12:00:00
3	Business	A Block	Jane Doe	3/15/2019, 12:00:00
4	Science	E Block	Bob Brown	5/5/2022, 12:00:00
5	Humanities	C Block	Alice White	8/20/2020, 12:00:00

View 2: Module Enrollment Statistics

Justification: Shows module popularity and success rates. Useful for curriculum planning and identifying modules needing support.

```
257
258 CREATE VIEW ModuleEnrollmentStats AS
259 SELECT m.moduleCode, m.moduleTitle, COUNT(e.matricNo) AS enrolledStudents,
260        SUM(CASE WHEN e.performance = 'Pass' THEN 1 ELSE 0 END) AS passedCount,
261        ROUND(AVG(CASE WHEN e.performance = 'Pass' THEN 100 ELSE 0 END), 2) AS passRate
262 FROM Module m
263 LEFT JOIN Enrollment e ON m.moduleCode = e.moduleCode
264 GROUP BY m.moduleCode, m.moduleTitle;
265
266 SELECT * FROM ModuleEnrollmentStats
```

	MODULECODE	MODULETITLE	ENROLLEDSTUDEN	PASSEDCOUNT	PASSRATE
1	M104	Networking	0	0	0
2	M105	Cyber Security	0	0	0
3	M101	Multi Media	2	1	50
4	M102	Databases	2	2	100
5	M103	Programming	1	1	100

View 3: Staff Teaching Load

Justification: Displays teaching workload per staff member. Useful for the head of department to balance teaching assignments.

```
245 -----view 3---
246 CREATE VIEW StaffTeachingLoad AS
247 SELECT a.staffNo, a.firstName || ' ' || a.lastName AS staffName,
248        COUNT(DISTINCT t.moduleCode) AS modulesTaught,
249        SUM(t.hoursPerWeek) AS totalWeeklyHours
250 FROM AcademicStaff a
251 LEFT JOIN TeachingSchedule t ON a.staffNo = t.staffNo
252 GROUP BY a.staffNo, a.firstName || ' ' || a.lastName;
253
254 SELECT * FROM StaffTeachingLoad
```

	STAFFNO	STAFFNAME	MODULESTAUGHT	TOTALWEEKLYHOL
1	S003	Bob Brown	2	15
2	S005	Charlie Black	1	9
3	S001	John Smith	1	5
4	S004	Alice White	1	10
5	S002	Jane Doe	0	(null)

View 4: Student Performance Summary

```
233
234 CREATE VIEW StudentPerformance AS
235 SELECT s.matricNo, s.firstName || ' ' || s.lastName AS studentName, c.courseTitle,
236        COUNT(e.moduleCode) AS modulesTaken,
237        SUM(CASE WHEN e.performance = 'Pass' THEN 1 ELSE 0 END) AS modulesPassed
238 FROM Student s
239 JOIN Course c ON s.courseCode = c.courseCode
240 JOIN Enrollment e ON s.matricNo = e.matricNo
241 GROUP BY s.matricNo, s.firstName || ' ' || s.lastName, c.courseTitle;
242
243 Select * from StudentPerformance;
```

	MATRICNO	STUDENTNAME	COURSE TITLE	MODULES TAKEN	MODULES PASSED
1	1001	Tom Jones	Pg Diploma IT	2	2
2	1002	Sarah Lee	Pg Diploma IT	2	1
3	1003	Mike Wong	MSc Computer Scien	1	1

Justification:

Shows each student's academic progress across their course. Useful for tutors tracking student performance.

4.3 Aggregated Columns

Column 1: Total Modules Passed

Tracks each student's academic progress without calculating every time. Useful for graduation eligibility checks.

```
UPDATE Student s SET totalModulesPassed = (
    SELECT COUNT(*)
    FROM Enrollment e
    WHERE e.matricNo = s.matricNo AND e.performance = 'Pass');
SELECT totalModulesPassed from Student
```

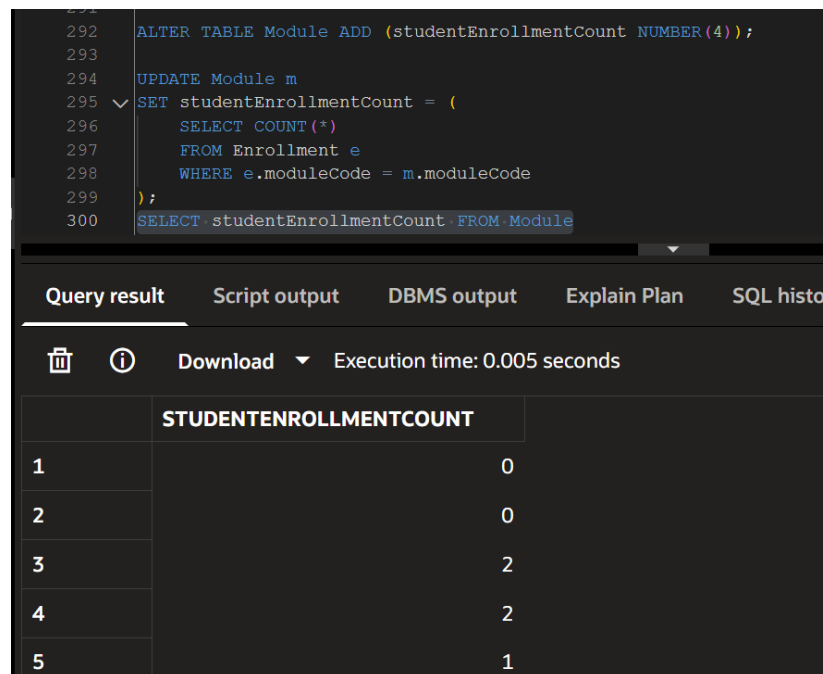
```
279 ALTER TABLE Student ADD (totalModulesPassed NUMBER(3));
280
281 UPDATE Student s
282 SET totalModulesPassed = (
283     SELECT COUNT(*)
284     FROM Enrollment e
285     WHERE e.matricNo = s.matricNo AND e.performance = 'Pass'
286 );
287
288 SELECT totalModulesPassed from Student
```

	TOTALMODULESPASSED
1	2
2	1
3	1
4	0
5	0

Column 2: Student Enrollment Count per Module

Shows module popularity at a glance. Helps the department plan class sizes and resource allocation.

```
UPDATE Module m SET studentEnrollmentCount = (  
    SELECT COUNT(*)  
    FROM Enrollment e  
    WHERE e.moduleCode = m.moduleCode);
```



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
292 ALTER TABLE Module ADD (studentEnrollmentCount NUMBER(4));  
293  
294 UPDATE Module m  
295 SET studentEnrollmentCount = (  
296     SELECT COUNT(*)  
297     FROM Enrollment e  
298     WHERE e.moduleCode = m.moduleCode  
299 );  
300 SELECT studentEnrollmentCount FROM Module
```

The results pane shows the output of the query, with columns labeled **STUDENTENROLLMENTCOUNT**. The results are as follows:

	STUDENTENROLLMENTCOUNT
1	0
2	0
3	2
4	2
5	1

Column 3: Total Teaching Hours per Staff

Calculates staff teaching workload. Useful for contract compliance and fair workload distribution.

```
UPDATE AcademicStaff a  
SET totalTeachingHours = ( SELECT SUM(t.hoursPerWeek)  
    FROM TeachingSchedule t  
    WHERE t.staffNo = a.staffNo);
```

304	ALTER TABLE academicStaff ADD (totalTeachingHours NUMBER(5));
305	
306	UPDATE academicStaff a
307	SET totalTeachingHours = (
308	SELECT SUM(t.hoursPerWeek)
309	FROM TeachingSchedule t
310	WHERE t.staffNo = a.staffNo
311	);
312	
313	SELECT totalTeachingHours FROM AcademicStaff

Query result	Script output	DBMS output	Explain Plan	SQL history
--------------	---------------	-------------	--------------	-------------

		Download	Execution time: 0.003 seconds	
--	--	----------	-------------------------------	--

	TOTALTEACHINGH
1	9
2	5
3	(null)
4	15
5	10

Column 4: Course Enrollment

Shows course popularity. Useful for marketing and resource planning.

```
UPDATE Course c SET totalEnrolledStudents = (
  SELECT COUNT(*) FROM Student s
  WHERE s.courseCode = c.courseCode);
```

314	ALTER TABLE Course ADD (totalEnrolledStudents NUMBER(4));
315	
316	UPDATE Course c
317	SET totalEnrolledStudents = (
318	SELECT COUNT(*)
319	FROM Student s
320	WHERE s.courseCode = c.courseCode);
321	SELECT totalEnrolledStudents FROM Course;

Query result	Script output	DBMS output	Explain Plan	SQL history
--------------	---------------	-------------	--------------	-------------

		Download	Execution time: 0.004 seconds	
--	--	----------	-------------------------------	--

	TOTALENROLLEDSTUDENTS
1	1
2	0
3	2
4	1
5	1

5. Part 4 – Database Application using Microsoft Access (Prototype)

5.1 Forms

frmDepartment

courseCode	courseTitle	durationYea	courseLeade
C002	Business Admi	4	S003

frmCourse

courseCode	courseTitle	durationYears	departmentName	courseLeaderStaffNo
C001	Computer Science	4	CIS	S002

frmModule

moduleCode	moduleTitle	startDate	endDate	texts	courseworkMarkPercentage	examMarkPercentage	courseCode	coordinatorStaffNo
M001	Database Systems	1/1/2024	6/1/2024	SQL Book	40	60	C001	S002

frmStudent

frmStudent

Student

matricNo	ST01	street	Street 1
userID	stu01	postCode	11564
firstName	Ahmed	dateOfBirth	5/1/2002
lastName	Al Faisal	sex	M
town	Riyadh	financialLoan	True
		courseCode	C001

moduleCode	performance						
M001	85						
*							

frmNextOfKin

frmNextOfKin

NextOfKin

nextOfKinID	1
name	Ali
address	Riyadh
phone	0501234567
relationship	Father
matricNo	ST01

frmAcademicStaff

frmAcademicStaff

AcademicStaff

staffNo	S001	sex	M
userID	u001	salary	\$12,000.00
firstName	Fahad	post	Manager
lastName	Al Saud	qualifications	PhD
phoneExtNo	101	address	Riyadh
officeNo	A12	departmentName	CIS

frmStudentModule

frmStudentModule

StudentModule

matricNo	ST01
moduleCode	M001
performance	85

frmTeaches



Teaches	
staffNo	S002
moduleCode	M001
hoursPerWeek	8

6. Part 5 – User Manual

The University Database Application is a relational database system designed to manage information about departments, courses, modules, academic staff, students, and next of kin. The application allows users to store, retrieve, and manipulate data efficiently for daily university operations.

Target Users: Academic staff, course coordinators, department heads, and administrative personnel.

Purpose: To provide a centralized system for tracking course enrollments, module performance, staff teaching loads, and student academic progress.

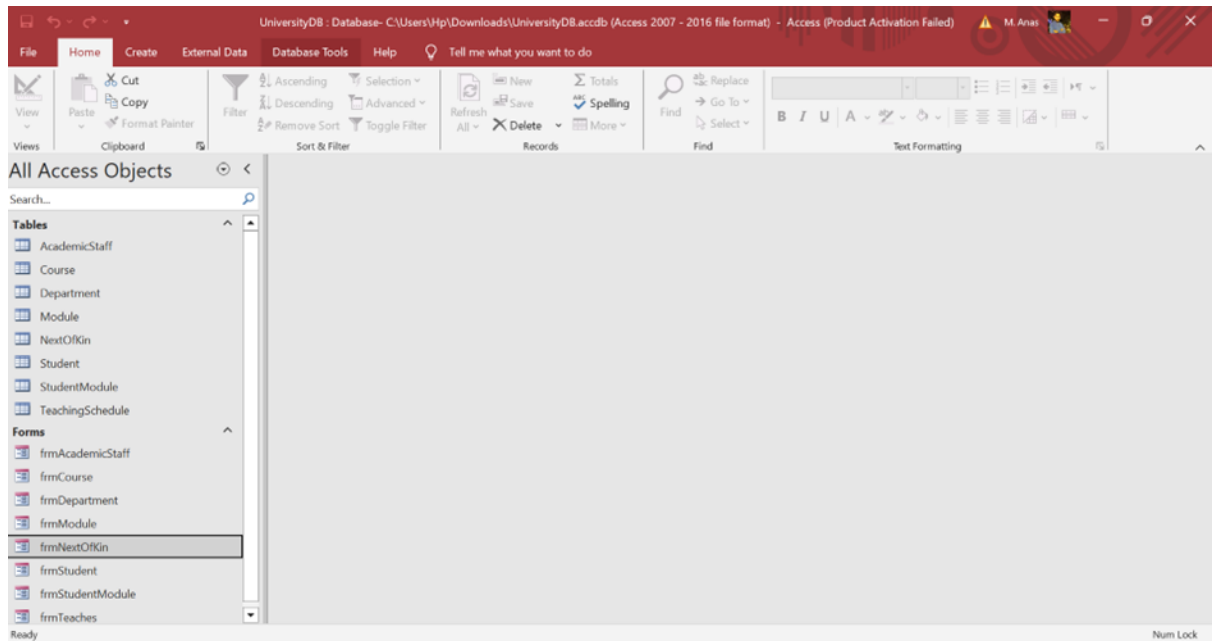
6.1 System Overview

The application consists of two versions. The SQL Schema runs on Oracle Database for backend data storage, query execution, and integrity enforcement. The User Interface runs on Microsoft Access providing frontend forms for data entry, navigation, and reporting.

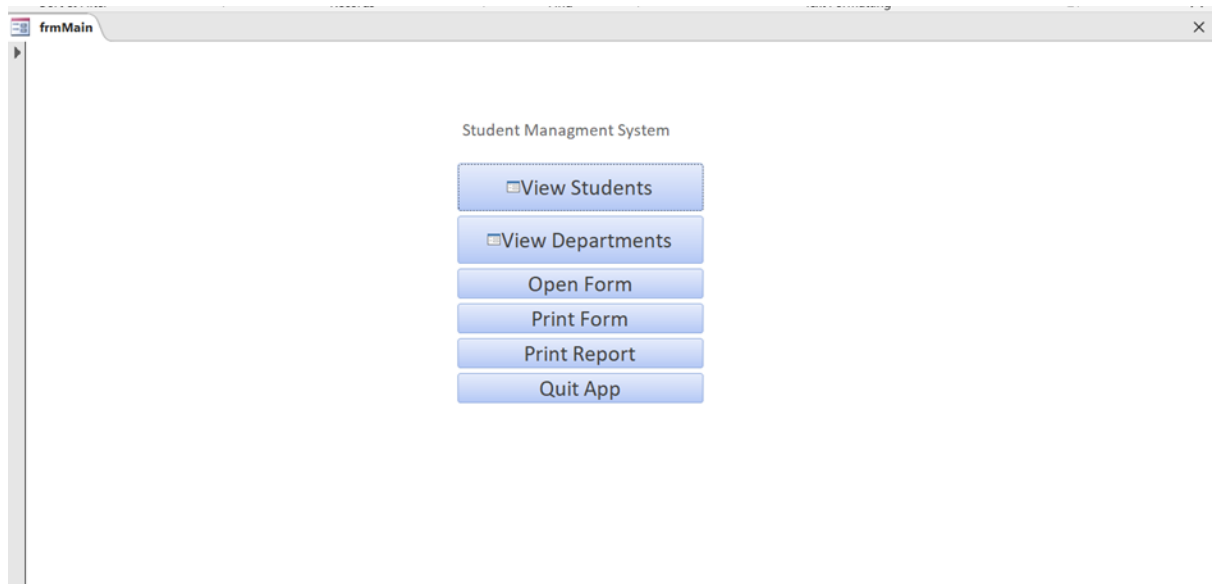
The main tables in the database include Department which stores academic departments and their managers, AcademicStaff which stores staff details including salaries and qualifications, Course which stores course information including duration and coordinator, Module which stores module details including dates and assessment percentages, Student which stores student personal details and enrollment information, NextOfKin which stores emergency contact information, Enrollment which tracks student module enrollment and pass/fail results, and TeachingSchedule which records staff teaching hours per module.

6.2 How to Use the System using Microsoft Access (for forms)

1. Open the database



2. Open the main form



3. Navigate between records using buttons

Student

matricNo: ST05 street: Street 5

userID: stu05 postCode: 11564

firstName: Yousef dateOfBirth:

lastName: Al Otaibi sex: M

town: Riyadh financialLoan: True

courseCode: C004

moduleCode	performance
M005	80
*	

Record: 1 of 5 of 5 No Filter Search

4. Add a new student using the Student Form

frmStudent

Student

matricNo: ST06 street: sbd

userID: Stu06 postCode: 39955

firstName: Dana dateOfBirth: 7/18/2005

lastName: sex: Female

town: Nyx financialLoan: False

courseCode:

5. Enroll a student in a module using the Enrollment Form

StudentModule

matricNo: ST06

moduleCode: M001

performance: 90

6. Assign Teaching Schedule

Teaches

staffNo S003

moduleCode M003

hoursPerWeek 6

7. Run queries (Query Wizard)

frmAcademicStaff | Module | TeachingSchedule | **AcademicStaff Query**

AcademicStaff

- staffNo
- userID
- firstName
- lastName
- phoneExtNo

Field: [staffNo] [userID] [firstName] [lastName] [departmentName]

Table: AcademicStaff AcademicStaff AcademicStaff AcademicStaff AcademicStaff

Sort:

Show: ☒ ☒ ☒ ☒ ☒

Criteria:

or:

Property Sheet

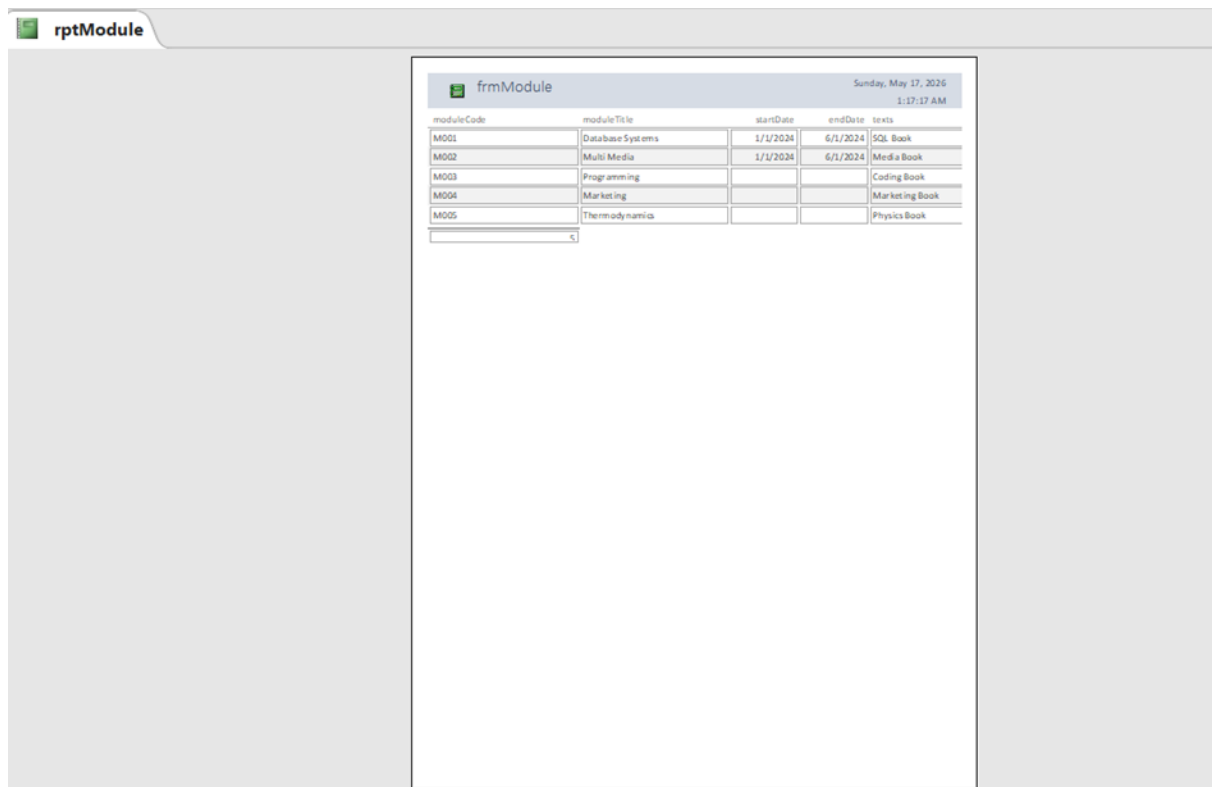
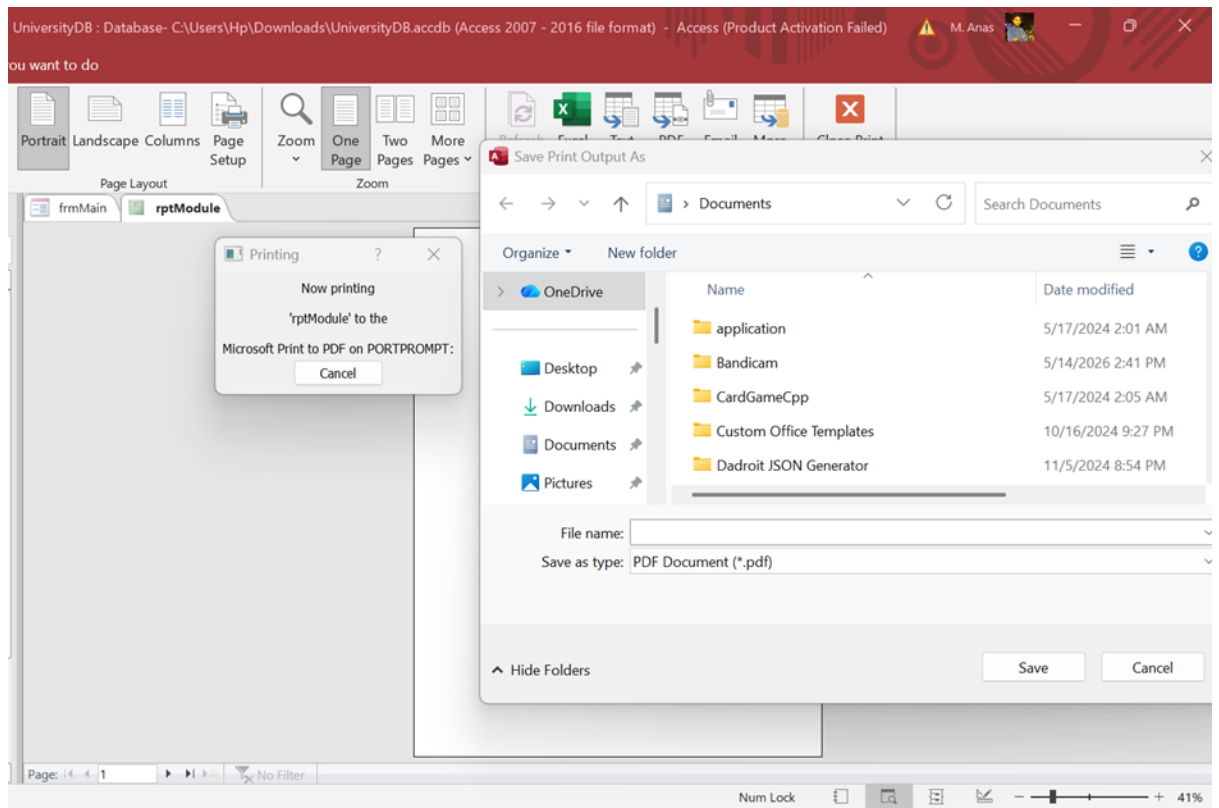
Selection type: Field Properties

General | **Lookup**

Description	
Format	
Input Mask	
Caption	
Text Format	

Num Lock SQL

8. View and print reports



6.3. SQL Database Manual

This manual describes how the University Database was built and how to maintain it. The database is implemented in Oracle SQL and stores information about departments, courses, modules, academic staff, students, next of kin, enrollments, and teaching schedules. This guide is intended for database administrators responsible for creating, populating, and maintaining the database.

Database Creation

The database consists of eight tables created in a specific order to handle foreign key dependencies. The tables were created using CREATE TABLE statements with primary keys defined inline.

The Department table was created first with departmentName as the primary key. The Course table was created next with courseCode as the primary key. The Module table was created with moduleCode as the primary key. The Student table was created with userID as the primary key and matricNo as an alternate key. The NextOfKin table was created with nextOfKinID as the primary key. The AcademicStaff table was created with staffNo as the primary key and userID as an alternate key. The Enrollment table was created with a composite primary key of matricNo and moduleCode. The TeachingSchedule table was created with a composite primary key of staffNo and moduleCode.

Foreign keys were added either inline during CREATE TABLE or using ALTER TABLE after all tables existed due to circular dependencies, specifically, between Department and AcademicStaff.

Inserting Data

Data was inserted in a specific order to respect foreign key constraints. The correct insertion order is Department first, then AcademicStaff, then NextOfKin, then Course, then Module, then Student, then Enrollment, then TeachingSchedule.

The circular dependency between Department and AcademicStaff required special handling. Department requires a staffNo foreign key referencing the manager in AcademicStaff, while AcademicStaff requires a departmentName foreign key referencing Department. To resolve this, Department records were inserted first with staffNo set to NULL. AcademicStaff records were inserted next. Finally, the Department table was updated with the correct staffNo values.

INSERT statements were used for all tables with approximately five to ten rows per table. For the Student table, the financialLoan column accepts BOOLEAN values of TRUE or FALSE with a default of FALSE. For the Enrollment table, the performance column accepts only Pass or Fail values enforced by a CHECK constraint.

Constraints and Integrity

Primary key constraints were defined during table creation on every table. Unique constraints were added to AcademicStaff.userID and Student.matricNo to enforce uniqueness.

Foreign key constraints were added either inline during CREATE TABLE or using ALTER TABLE after all tables existed. The AcademicStaff table references Department through departmentName. The Department table references AcademicStaff through staffNo. The Course table references AcademicStaff through staffNo and Department through departmentName. The Module table references AcademicStaff through staffNo and Course through courseCode. The Student table references NextOfKin through nextOfKinID and Course through courseCode. The Enrollment table references Student through matricNo and Module through moduleCode. The TeachingSchedule table references AcademicStaff through staffNo and Module through moduleCode.

Check constraints were added to validate data values. For the AcademicStaff and Student tables, a check constraint restricts sex to M or F only. For the Module table, check constraints restrict courseworkMarkPercentage and examMarkPercentage to values between 0 and 100. For the TeachingSchedule table, a check constraint restricts hoursPerWeek to values between 1 and 40.

Indexes were created on foreign key columns to improve query performance. The following indexes were added using CREATE INDEX statements:

1. idx_enroll_module on the moduleCode column of the Enrollment table
2. idx_teach_module on the moduleCode column of the TeachingSchedule table
3. idx_student_course on the courseCode column of the Student table
4. idx_module_course on the courseCode column of the Module table
5. idx_academic_department on the departmentName column of the AcademicStaff table

These indexes speed up JOIN operations between tables by allowing the database to locate foreign key values more efficiently.

Queries

Four views were created to simplify common queries.

1. The DepartmentSummary view joins Department and AcademicStaff to display department names, locations, manager names, and manager start dates.
2. The StudentPerformance view joins Student, Course, and Enrollment to display each student's matriculation number, name, course title, total modules taken, and total modules passed.
3. The StaffTeachingLoad view joins AcademicStaff and TeachingSchedule to display each staff member's number, name, modules taught, and total weekly teaching hours.
4. The ModuleEnrollmentStats view joins Module and Enrollment to display module code, module title, number of enrolled students, number of students who passed, and pass rate percentage.

Four derived columns were added to store pre-calculated aggregated data. These columns improve query performance by eliminating repetitive COUNT and SUM operations.

1. For the Student table, a column named totalModulesPassed was added using ALTER TABLE. The column was populated using an UPDATE statement with a subquery that counts Pass records from the Enrollment table for each student.
2. For the Module table, a column named studentEnrollmentCount was added using ALTER TABLE. The column was populated using an UPDATE statement with a subquery that counts enrollment records from the Enrollment table for each module.
3. For the AcademicStaff table, a column named totalTeachingHours was added using ALTER TABLE. The column was populated using an UPDATE statement with a subquery that sums hoursPerWeek from the TeachingSchedule table for each staff member.
4. For the Course table, a column named totalEnrolledStudents was added using ALTER TABLE. The column was populated using an UPDATE statement with a subquery that counts students from the Student table for each course.

Maintenance

Query results and entire tables can be exported from Oracle SQL Developer by right-clicking the result grid or table name and selecting Export, then choosing CSV or Excel format.

- If a foreign key error occurs when inserting data, ensure the parent record exists before inserting the child record.

- If a check constraint error occurs, verify the data value falls within the allowed range. Sex must be M or F. Percentages must be between 0 and 100. Hours per week must be between 1 and 40.
- If a unique constraint error occurs, ensure the value being inserted does not already exist in the column. UserID and matricNo must be unique.
- If a view returns no data, check that the underlying tables contain data and that JOIN conditions are correctly matched.

7. Work Distribution

Name	Contribution
Dana	Database design, ER diagram, Normalization
Rafal Alqahtani	ER Diagram, Implementation, Tables, Forms
Daman	ER Diagram, Queries, Tables, Documentation

8. Part 7 – Individual Critical Evaluation

Example Structure:

- What I learned
- Challenges faced
- Skills developed
- Relevance to my studies and career

8.1 Critical Evaluation by Dana Alzahid - 202100121

This coursework provided me with a comprehensive and hands-on experience in designing and implementing a relational database system from scratch. Working through each phase of the database development lifecycle, from requirements analysis and ER modelling to SQL implementation and querying therefore it deepened my understanding of how theoretical database concepts translate into real-world systems.

During the design phase, I was responsible for producing the ER diagram and relational schema. This process reinforced my understanding of how to identify entities, attributes, and relationships from a written specification, and how to translate those into a structured UML class diagram. Deciding how to handle many-to-many relationships, such as

the student-module enrollment and the staff-module teaching relationship, made it clear why associative entities like Enrollment and TeachingSchedule are necessary in relational database design.

Applying normalization was one of the most valuable parts of the coursework. Working through the given badly-structured relation and decomposing it step by step from 1NF through to 3NF made the purpose of normalization much clearer to me.

In particular, identifying the transitive dependency between the student's flat number and address, and splitting it into a separate Flat relation, demonstrated how normalization eliminates redundancy and prevents data anomalies that would otherwise cause problems during insertion, deletion, and updating of records. One of the more challenging aspects of the design was correctly modelling the optional management roles for academic staff. The specification stated that a member of staff may not be assigned any management role at all, which meant the cardinality of certain relationships needed to reflect optionality rather than a mandatory assignment. This required careful reading of the requirements and highlighted how easily cardinality errors can be introduced if the specification is not analysed precisely.

A significant limitation we encountered during implementation was introduced by Oracle SQL itself. In our relational schema, we specified On Delete No Action On Update Cascade for all foreign key relationships, which is the standard notation taught in the module and reflects the theoretically correct design. However, Oracle SQL does not support the ON UPDATE CASCADE clause on foreign key constraints, unlike other database management systems such as MySQL or Microsoft SQL Server. As a result, our Oracle implementation could only enforce ON DELETE NO ACTION, which is the closest supported behaviour available in Oracle. This was not an error in our design but rather a platform limitation. Our relational schema remains correct at the theoretical level, and the deviation in the SQL implementation was a conscious and informed decision made in response to Oracle's constraints. A further limitation of Oracle that affected our implementation was the handling of the BOOLEAN data type.

Oracle SQL does not natively support BOOLEAN as a column type in table definitions, unlike other database systems. For the financialLoan column in the Student table, we used a workaround by defining the column with a CHECK constraint restricting values to 0 and 1, or alternatively using a CHAR(1) field with Y and N values. This is a well-known Oracle limitation that required adaptation from the ideal design.

In Microsoft Access, we were able to represent this column more naturally as a Yes/No field, which better reflects the original requirement. In terms of how this coursework

relates to my broader studies, the database design process connects closely to the Systems Analysis and Design module, where requirements are gathered and translated into system models. The discipline of moving from a written requirement to a formal schema mirrors the same process of moving from a business requirement to a system specification. Additionally, the SQL implementation skills developed throughout this project are directly relevant to software development roles, where database interaction is a core competency.

Overall, this project has strengthened my confidence in database design and has given me practical experience that I believe will be directly applicable in future employment. The ability to design a normalized schema, implement it in SQL, and query it effectively are skills that are in demand across a wide range of technology roles. Encountering real platform limitations such as those in Oracle also prepared me to think critically about the gap between theoretical database design and practical implementation, which is an important skill for any future role involving database systems.

8.2 Evaluation by Rafal Alqahtani - 202200062

This coursework gave me practical experience in the full process of database design and implementation. My main contributions were the ER diagram, table creation in both Microsoft Access and Oracle SQL, and building the data entry forms for the prototype application. One of the most valuable things I learned was the importance of insertion order when dealing with foreign key constraints.

When implementing the tables in Oracle, I had to carefully plan which tables were created and populated first to avoid referential integrity errors. The circular dependency between the Department and AcademicStaff tables was a particular challenge that required inserting Department records with a NULL manager value first, then updating them once the staff records were in place. I also encountered limitations specific to Oracle SQL during implementation.

Oracle does not support ON UPDATE CASCADE on foreign key constraints, which meant our implementation differed slightly from the relational schema we designed. This was not a mistake in our design but a known Oracle restriction, and our schema remains correct at the theoretical level. Building the Microsoft Access forms helped me understand how a database application works from the user's perspective, and how the front-end connects to the underlying tables through relationships and foreign keys.

Regarding the System Features and Automation sections, these were not fully developed in the final submission. The coursework specification for Part 4 required the implementation of a prototype database application but did not specify Microsoft Access as the required platform, nor did it define any particular automation features or navigation requirements. Since Microsoft Access was our own chosen tool and the specification did not mandate specific features beyond facilitating the creation, maintenance, and querying of records, we did not expand these sections further. The decision was therefore based on the absence of explicit requirements from the module instructor rather than an oversight on our part. The core database functionality was fully implemented through the forms, which allow records to be added, updated, and searched using the standard Microsoft Access interface.

Overall, this project improved my practical SQL skills and my understanding of how database design decisions affect real implementation. The experience is directly relevant to future roles in software development and data management.

8.3 Critical Evaluation by Daman Arshad - 202300481

During this project, I applied the SQL and database concepts I learnt during my coursework, particularly the Systems Analysis and Design and the Object Oriented Analysis and Design concepts, to the requirements of this project. This included integrating database concepts, designing and implementing the database along with my group members. The project cleared the purpose of normalisation in databases and the extensive depth that goes into designing them for organizations.

During implementation with SQL, I identified a circular dependency between the Department and AcademicStaff tables. Department requires a staffNo foreign key referencing the manager, while AcademicStaff requires a departmentName foreign key referencing the department, creating a chicken-and-egg problem where neither table could be inserted first. To resolve this, I ensured that Department.staffNo was nullable, inserted Department records first with NULL values, inserted AcademicStaff records next, then updated Department with the correct staffNo. This works well as a new Department can be added to the database so that the university can start planning it before hiring staff. This order of insertion reinforces referential integrity, ensuring the parent record exists before adding the child record.

Moreover, we ran into a challenge while trying to modify the constraints to add behaviour constraints like ON UPDATE CASCADE for the foreign keys. However, Oracle FreeSQL software does not support these clauses, leading to a practical limitation.

In a production environment with larger datasets, a better solution would be to use DEFERRABLE constraints or remove the circular dependency by moving the manager relationship to AcademicStaff as an isHeadOfDepartment flag. This experience reinforced my understanding of how real-world database design requires balancing ideal normalization against practical implementation constraints.

Additionally, the database could be further enhanced by adding a separate table for staff with management positions in case the university is huge and has a lot more managers. The current database works as per the instructions, but may need more work before being launched in a practical capacity with larger datasets.

Overall, the project helped me learn how working with real database applications can be complicated and complex. It improved my database skills which can potentially help me during future roles involving databases. It compiles all important concepts practically and improves my understanding.

9. Conclusion

This project demonstrated the full lifecycle of database development, including design, implementation, and querying. It highlights the importance of structured data management in real-world systems.

The database successfully delivered a fully functional relational database system for a university. All requirements from the instructions were followed, including eight normalized tables, primary and foreign keys, check constraints, indexes, views, aggregated columns, and ten SQL queries. The database was implemented in both Oracle SQL and Microsoft Access, with a front-end application providing data entry forms and navigation. The project reinforced the importance of normalization, referential integrity, and careful insertion ordering when dealing with foreign key dependencies. Despite platform limitations such as Oracle's lack of ON UPDATE CASCADE support, the final system effectively manages departments, courses, modules, staff, students, and enrollments for academic operations.